

Android 传感器数据采集实验报告

一、实验名称

Android 本地传感器数据采集系统开发

二、实验目的

掌握 Android 后台 Service 后台常驻采集技术；
熟练使用 SQLite 实现本地数据持久化存储；
完成列表展示、数据筛选、CSV 文件导出功能；
掌握 Android 文件存储、通知权限、界面交互开发流程；
理解 Android 项目分层架构，独立完成完整可运行 App。

三、开发环境

开发工具：Android Studio Hedgehog

编译 Gradle：7.4

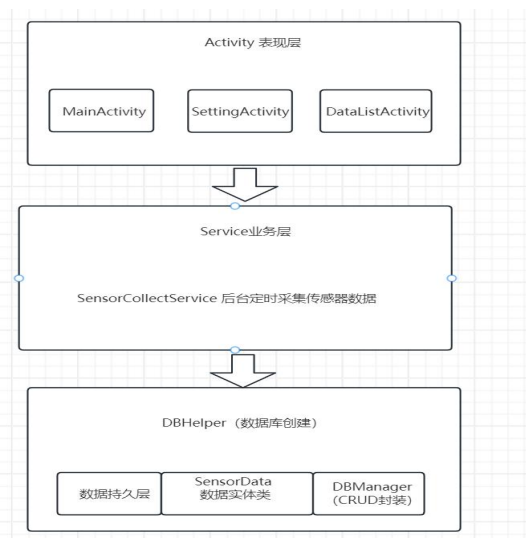
编译 SDK：API 34

最低兼容 SDK：26

编程语言：Java

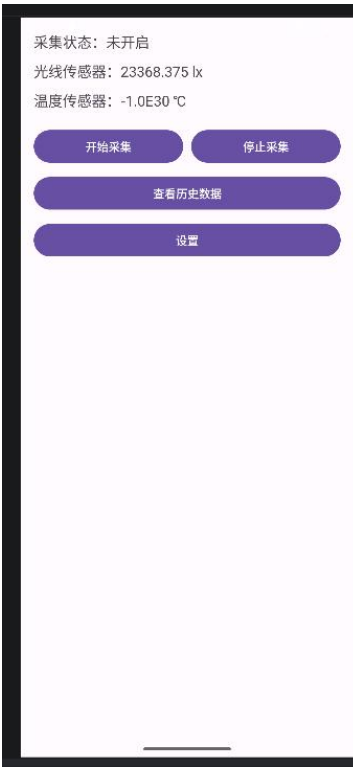
数据库：原生 SQLite

四、项目整体架构图



五、程序运行截图展示

首页主界面



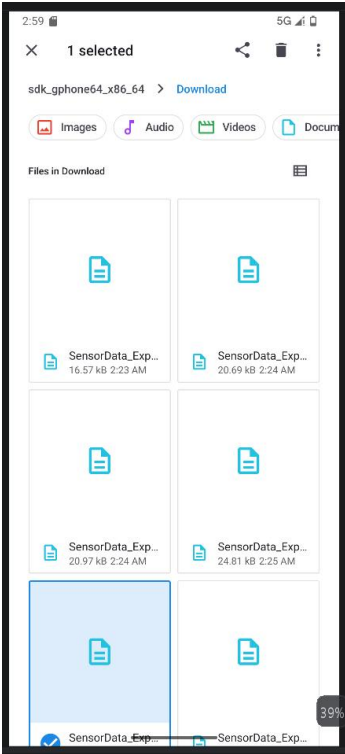
参数设置界面



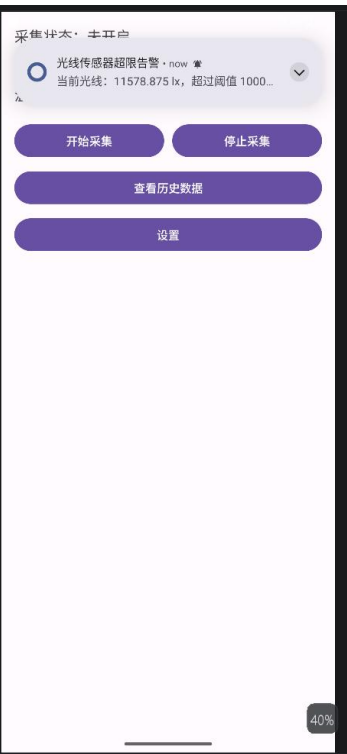
历史数据列表筛选界面



CSV 导出



阈值超限报警



六、项目全部核心代码

DBHelper & DBManager 数据库模块

```
1 usage
public DBHelper(Context context) { super(context, DB_NAME, factory, null, DB_VERSION); }

@Override
public void onCreate(SQLiteDatabase db) {
    String createSql = "CREATE TABLE " + TABLE_SENSOR_DATA + " ("
        + COL_ID + " INTEGER PRIMARY KEY AUTOINCREMENT,"
        + COL_TIME + " TEXT,"
        + COL_TYPE + " TEXT,"
        + COL_VALUE + " REAL)";
    db.execSQL(createSql);
}

10 usages
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_SENSOR_DATA);
    onCreate(db);
}

1 usage
public List<SensorData> queryAll() {
    List<SensorData> list = new ArrayList<>();
    Cursor cursor = db.query(DBHelper.TABLE_SENSOR_DATA, columns, null, selection, null, selectionArgs, null);
    if (cursor.moveToFirst()) {
        do {
            SensorData data = new SensorData();
            data.setId(cursor.getLong(columnIndex 0));
            data.setTimestamp(cursor.getString(columnIndex 1));
            data.setSensorType(cursor.getString(columnIndex 2));
            data.setValue(cursor.getFloat(columnIndex 3));
            list.add(data);
        } while (cursor.moveToNext());
    }
    cursor.close();
    return list;
}
```

后台采集服务 SensorCollectService

```
1 usage
@Override
public void onCreate() {...}

2 usages
private void loadConfig() {...}

1 usage
private void registerSensor() {...}

2 usages
public void startCollect() {...}

2 usages
public void stopCollect() {...}
```

历史页面 CSV 导出 DataListActivity

```
private void exportDataToCsv() {
    List<SensorData> dataList = dbManager.queryAll();
    if (dataList.isEmpty()) {
        Toast.makeText(context, this, "暂无数据可导出", Toast.LENGTH_SHORT).show();
        return;
    }

    StringBuilder csvContent = new StringBuilder();
    csvContent.append("采集时间,传感器类型,传感器数值\n");
    for (SensorData data : dataList) {
        String time = data.getTimestamp().replace(" ", ",");
        String type = data.getSensorType().replace(" ", ",");
        String value = String.valueOf(data.getValue()).replace(" ", ",");
        csvContent.append(time).append(",").append(type).append(",").append(value).append("\n");
    }

    SimpleDateFormat sdf = new SimpleDateFormat("yyyyMMdd_HH:mm:ss", Locale.getDefault());
    String fileName = "SensorData_Export_" + sdf.format(new Date()) + ".csv";

    // API29及以上用MediaStore, 低版本走旧File方式
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
        ContentValues values = new ContentValues();
        values.put(MediaStore.Downloads.DISPLAY_NAME, fileName);
        values.put(MediaStore.Downloads.MIME_TYPE, "text/csv");
        values.put(MediaStore.Downloads.RELATIVE_PATH, Environment.DIRECTORY_DOWNLOADS);

        Uri uri = getContentResolver().insert(MediaStore.Downloads.EXTERNAL_CONTENT_URI, values);
        if (uri == null) {
            Toast.makeText(context, this, "文件创建失败", Toast.LENGTH_SHORT).show();
            return;
        }

        try (OutputStream outputStream = getContentResolver().openOutputStream(uri)) {
            outputStream.write(csvContent.toString().getBytes(StandardCharsets.UTF_8));
            outputStream.flush();
            showExportSuccessNotification(uri, fileName);
            Toast.makeText(context, this, "导出成功！下拉通知栏点击打开", Toast.LENGTH_SHORT).show();
        } catch (IOException e) {
            e.printStackTrace();
            Toast.makeText(context, this, "导出失败: " + e.getMessage(), Toast.LENGTH_SHORT).show();
        }
    } else {
        // 低版本走旧File方式
        File file = new File(Environment.getExternalStorageDirectory(), fileName);
        try (FileWriter writer = new FileWriter(file)) {
            writer.write(csvContent.toString());
        } catch (IOException e) {
            e.printStackTrace();
            Toast.makeText(context, this, "导出失败: " + e.getMessage(), Toast.LENGTH_SHORT).show();
        }
    }
}
```

MainActivity.java

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    bindView();
    dbManager = new DBManager(context);
    bindService(new Intent(packageContext, SensorCollectService.class), conn, Context.BIND_AUTO_CREATE);

    // 动态注册电池广播接收器
    batteryReceiver = new BatteryReceiver();
    IntentFilter filter = new IntentFilter(Intent.ACTION_BATTERY_CHANGED);
    registerReceiver(batteryReceiver, filter);

    // ===== 说明: 申请通知权限 (Android13+ 需于系统设置页面通知) =====
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.POST_NOTIFICATIONS},
            REQUEST_CODE);
    }

    // MainActivity onCreate 末尾添加
    // 该服务后台采集开关, 自动恢复采集状态
    SharedPreferences sp = getSharedPreferences(SettingActivity.SP_NAME, MODE_PRIVATE);
    boolean autoStartCollect = sp.getBoolean(SettingActivity.KEY_BG_ENABLE, false);
    if (autoStartCollect) {
        Intent autoServiceIntent = new Intent(packageContext, SensorCollectService.class);
        startService(autoServiceIntent);
    }
}
```

七、个人总结

本次实验独立完成了一套完整 Android 传感器采集 App，完整覆盖界面交互、后台服务、SQLite 数据库、文件导出、系统通知等核心知识点。

- 1、掌握了 Service 后台常驻运行机制，理解生命周期管理；
- 2、熟练使用 SQLite 完成本地数据增删改查，实现大数据列表分页 / 筛选；
- 3、学习 Android 分区存储适配方案，解决不同安卓版本文件读写兼容问题；
- 4、学会规范项目结构，分层开发降低代码耦合，同时完成 GitHub 仓库标准化 README 文档、Debug 安装包打包；
- 5、存在不足：未完成趋势图表与前台保活功能，后续可引入 MPAndroidChart 绘制折线图，改造服务为前台服务提升后台保活能力。